

EXP NO: 1

THREE-LAYER NEURAL NETWORK FROM SCRATCH

AIM

To design and implement a three-layer neural network from scratch using Python and NumPy, train it using the backpropagation algorithm, and classify handwritten digits from the MNIST dataset.

OBJECTIVES

1. To understand the structure of a basic feedforward neural network.
2. To implement forward propagation and backpropagation from scratch.
3. To use the sigmoid activation function.
4. To calculate the cross-entropy loss.
5. To train the neural network using gradient descent.
6. To visualize the training loss and predict handwritten digits.

SOFTWARE REQUIREMENTS

- Google Colab
- Python 3.x
- NumPy
- Matplotlib
- TensorFlow
- Scikit-learn

THEORY

A three-layer neural network consists of:

Input Layer → Hidden Layer → Output Layer

For MNIST:

- Input layer = **784 neurons** because each image is 28×28 pixels.
- Hidden layer = **64 neurons**.
- Output layer = **10 neurons** representing digits 0 to 9.

The network performs:

Forward Propagation → Loss Calculation → Backpropagation → Weight Update

The sigmoid activation function is:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

The weights are updated using gradient descent:

$$W = W - \eta \partial W \partial L$$

where η is the learning rate.

ALGORITHM

1. Import NumPy, Matplotlib, TensorFlow and Scikit-learn libraries.
2. Load the MNIST handwritten digit dataset.
3. Select the training images and labels.
4. Reshape each 28×28 image into a 784-element vector.
5. Normalize pixel values between 0 and 1.
6. Convert the target labels into one-hot encoded form.
7. Initialize weights and biases randomly.
8. Perform forward propagation.
9. Calculate the sigmoid activation.
10. Calculate the loss using cross-entropy.
11. Perform backpropagation to calculate gradients.
12. Update weights and biases using gradient descent.
13. Repeat the process for several epochs.
14. Plot the training loss.
15. Predict a sample handwritten digit.
16. Display the actual and predicted digit.

PROGRAM

```
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import mnist
from sklearn.preprocessing import OneHotEncoder

(X_train_original, y_train_original), (_, _) = mnist.load_data()

X_train_original = X_train_original[:10000]
y_train_original = y_train_original[:10000]

X_train = X_train_original.reshape(-1, 784) / 255.0

encoder = OneHotEncoder(sparse_output=False)
y_train = encoder.fit_transform(y_train_original.reshape(-1, 1))

input_size = 784
hidden_size = 64
output_size = 10
```

```

np.random.seed(42)

W1 = np.random.randn(input_size, hidden_size) * 0.01
b1 = np.zeros((1, hidden_size))

W2 = np.random.randn(hidden_size, output_size) * 0.01
b2 = np.zeros((1, output_size))

def sigmoid(x):
    return 1 / (1 + np.exp(-np.clip(x, -500, 500)))

def sigmoid_deriv(x):
    return x * (1 - x)

def loss_fn(y, y_hat):
    return -np.mean(y * np.log(y_hat + 1e-8))

epochs = 10
lr = 0.1
losses = []

for epoch in range(epochs):
    total_loss = 0

    for i in range(X_train.shape[0]):
        x = X_train[i:i+1]
        y = y_train[i:i+1]

        z1 = x @ W1 + b1
        a1 = sigmoid(z1)

        z2 = a1 @ W2 + b2
        a2 = sigmoid(z2)

        loss = loss_fn(y, a2)
        total_loss += loss

        dz2 = a2 - y
        dW2 = a1.T @ dz2
        db2 = dz2

        dz1 = (dz2 @ W2.T) * sigmoid_deriv(a1)
        dW1 = x.T @ dz1
        db1 = dz1

        W2 -= lr * dW2
        b2 -= lr * db2

```

```
W1 -= lr * dW1
b1 -= lr * db1
```

```
losses.append(total_loss / X_train.shape[0])
print(f"Epoch {epoch + 1}, Loss: {losses[-1]:.4f}")
```

```
plt.plot(losses, marker='o')
plt.title("Training Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.grid(True)
plt.show()
```

```
def predict(img):
    img = img.reshape(1, 784) / 255.0
    a1 = sigmoid(img @ W1 + b1)
    a2 = sigmoid(a1 @ W2 + b2)
    return np.argmax(a2)
```

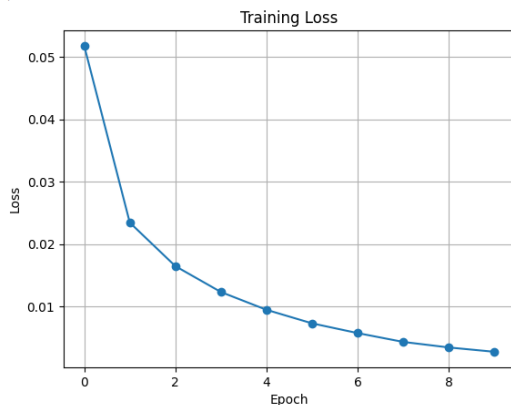
```
idx = 100
```

```
prediction = predict(X_train_original[idx])
```

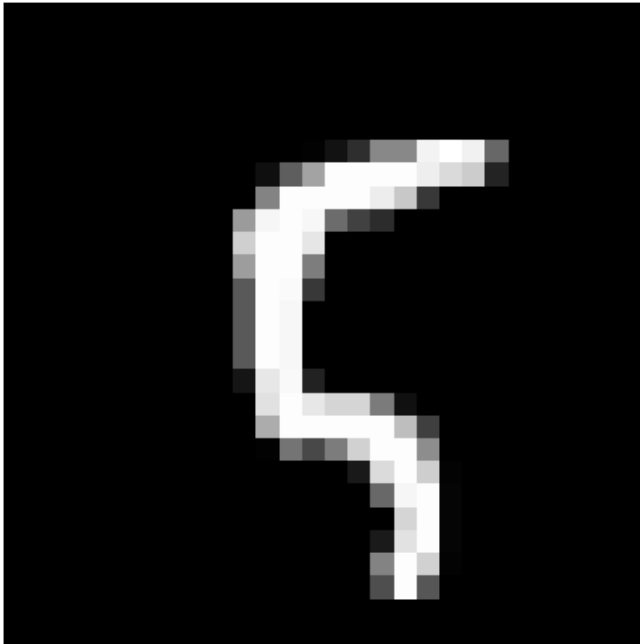
```
plt.imshow(X_train_original[idx], cmap='gray')
plt.title(f"Actual: {y_train_original[idx]} | Prediction: {prediction}")
plt.axis('off')
plt.show()
```

```
print("Actual Digit:", y_train_original[idx])
print("Predicted Digit:", prediction)
```

```
Epoch 1, Loss: 0.0518
Epoch 2, Loss: 0.0235
Epoch 3, Loss: 0.0165
Epoch 4, Loss: 0.0123
Epoch 5, Loss: 0.0095
Epoch 6, Loss: 0.0073
Epoch 7, Loss: 0.0057
Epoch 8, Loss: 0.0043
Epoch 9, Loss: 0.0034
Epoch 10, Loss: 0.0028
```



Actual: 5 | Prediction: 5



Actual Digit: 5
Predicted Digit: 5

RESULT

The three-layer neural network was successfully implemented from scratch using Python and NumPy. The network was trained on the MNIST handwritten digit dataset using forward propagation, sigmoid activation, cross-entropy loss, backpropagation, and gradient descent. The training loss was visualized, and the trained model successfully predicted handwritten digits.